

网络层（数据平面）

1. 导论

1.1 网络层服务

- 在发送主机和接收主机对之间传送段
- 在发送端将段封装到数据报中
- 在接收端，将段上交给传输层实体
- 网络层协议存在于每一个主机和路由器
- 路由器检查每一个经过它的IP数据报的头部

1.2 网络层的功能

- **转发：**将分组从路由器的输入接口转发到合适的输出接口
- **路由：**使用路由算法来决定分组从发送主机到目标接收主机的路径
 - 路由选择算法
 - 路由选择协议

旅行的类比：

转发：通过单个路口的过程

路由：从源到目的的路由路径规划过程

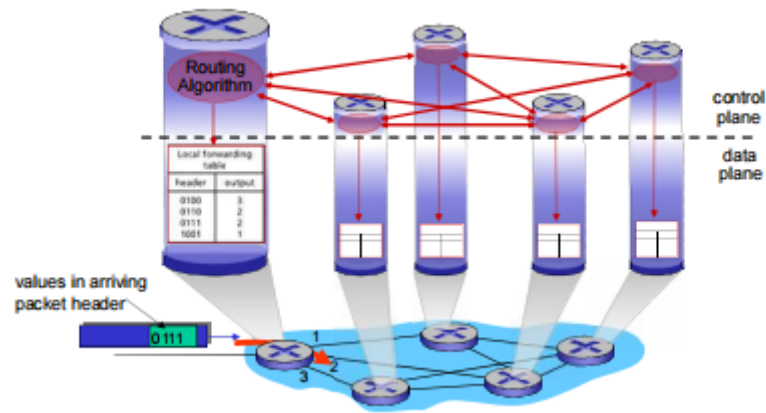
1.3 网络层的平面

- 数据平面：
 - 本地，每个路由器功能
 - 决定从路由器输入端口到达的分组如何转发到输出端口
 - 转发功能：传统方式：基于目标地址+转发表
SDN方式：基于多个字段+流表
- 控制平面：
 - 网路范围的逻辑
 - 决定数据报如何在路由器之间路由，决定数据报从源到目标主机之间的端到端路径
 - 2个控制平面的方法：
传统的路由算法：在路由器中实现
SDN：在远程的服务器中实现

1.4 传统方式

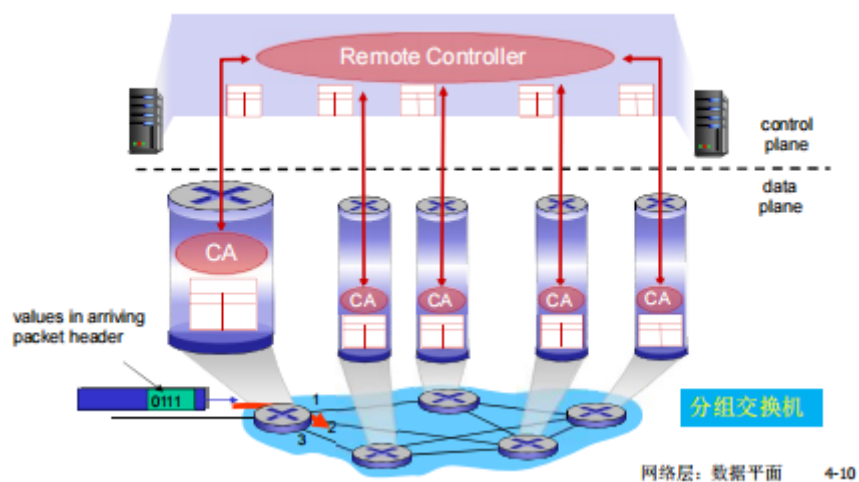
- 在每一个路由器中的单独路由器算法元件，在控制平面进行交互
- 路由和转发的相互作用：
控制平面：路由算法决定端到端路径
数据平面：IP协议根据转发表决定了IP数据报在此路由器上的局部转发

- 传统方式是分布式的



1.5 SDN方式

- SDN: 逻辑集中的
- 一个不同的（通常是远程的）控制器与本地控制代理（CA）交互



1.6 网络服务模型

- 对于单个数据报的服务：可靠传送；延迟保证
- 对于数据报流的服务：
 - 保序数据报传送
 - 保证流的最小带宽
 - 分组之间的延迟差

网络层服务模型：

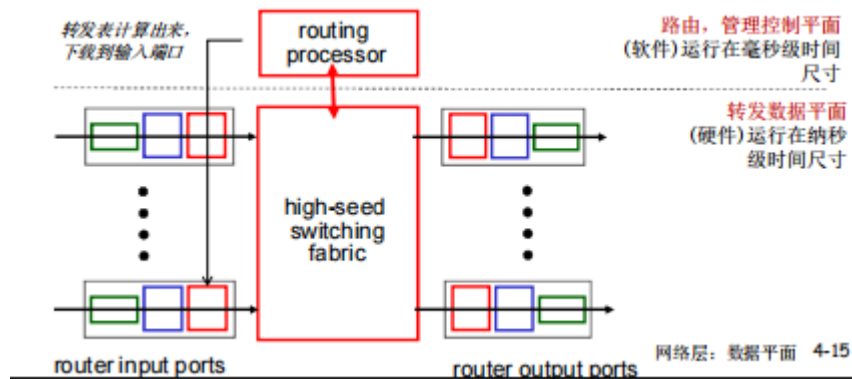
网络架构	服务模型	保证？				拥塞反馈
		带宽	丢失	保序	延迟	
Internet	best effort	none	no	no	no	no (inferred via loss)
ATM	CBR 恒定速率	constant rate	yes	yes	yes	no congestion
ATM	VBR 变化速率	guaranteed rate	yes	yes	yes	no congestion
ATM	ABR 可用比特率	guaranteed minimum	no	yes	no	yes
ATM	UBR 不指名比特率	none	no	yes	no	no

网络层和传输层连接服务的区别：

- 网络层：在2个主机之间，涉及到路径上的一些路由器
- 传输层：在2个进程之间，很可能只体现在端系统上

2.路由器组成

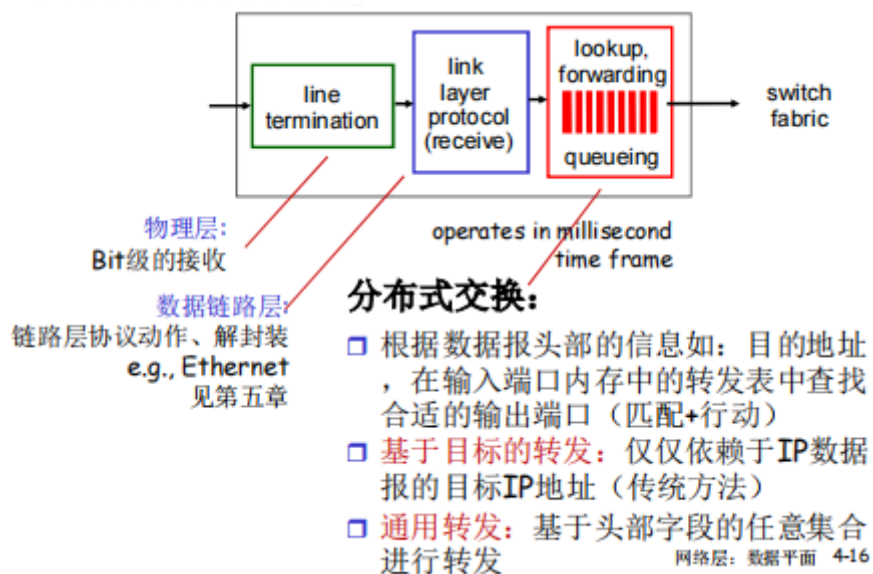
2.1 路由器结构



高层通用路由器体系架构：

- 路由：运行路由选择算法/协议生成路由表
- 转发：从输入到输出链路交换数据报，根据路由表进行分组的转发

2.2 输入端口功能

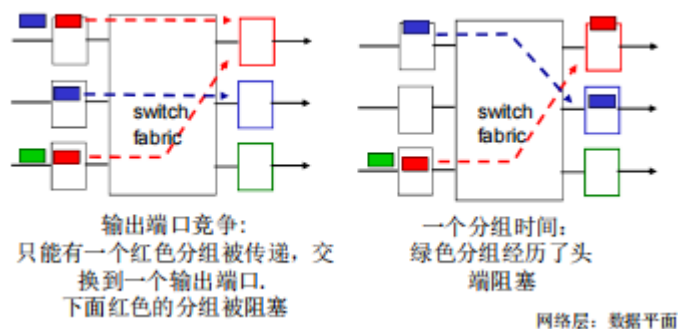


当给定目标地址查找转发表时, 采用最长地址前缀匹配的目标地址表项。

2.3 输入端口缓存

当交换机构的速率小于输入端口的汇聚速率时→在输入端口可能要排队

Head - of - the - Line (HOL) blocking: 排在队头的数据报阻止了队列中其他数据报向前移动



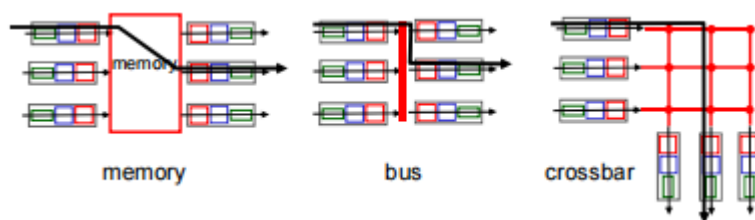
交换结构: switch fabric

将分组从输入缓冲区传输到合适的输出端口

交换速率: 分组可以按照该速率从输入传输到输出

- 运行速度经常是输入/输出链路速率的若干倍
- N个输入端口, 交换机构的交换速度是输入线路速度的N倍比较理想, 才不会成为瓶颈

3种典型的交换机构:



2.4 交换结构

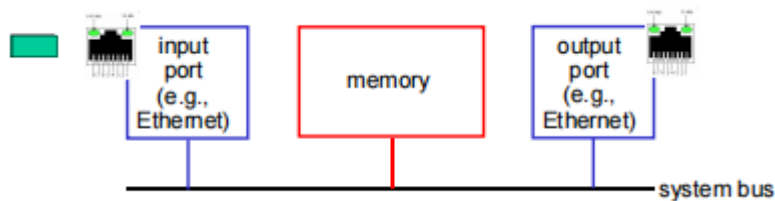
2.4.1 通过内存交换

在CPU直接控制下的交换，采用传统的计算机

分组被拷贝到系统内存，CPU从分组的头部提取出目标地址，查找转发表，找到对应的输出端口，拷贝到输出端口

转发速率被内存的带宽限制（数据报通过BUS两遍）

一次只能转发一个分组



2.4.2 通过总线交换

数据报通过共享总线，从输入端口转发到输出端口

总线竞争：交换速度受限于总线带宽

1次处理一个分组

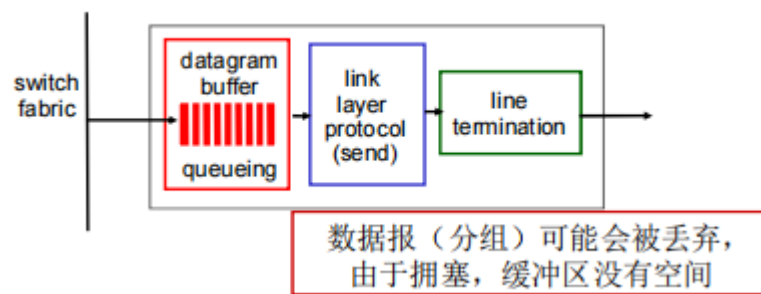
2.4.3 通过互联网络的交换

同时并发转发多个分组，克服总线带宽限制

Banyan网络，crossbar和其他的互联网络被开发，将多个处理器连接成多处理器

将数据报分片为固定长度的信元，通过交换网络交换

2.5 输出端口



- 当数据报从交换机构的到达速度比传输速率快就需要输出端口缓存
- 由调度规则选择排队的数据报进行传输

2.6 调度机制

- 调度：选择下一个要通过链路传输的分组

2.6.1 FIFO scheduling

按照分组的次序发送

丢弃策略：

- tail drop: 丢弃刚到达的分组
- priority: 根据优先权/移出分组
- random: 随机地丢弃/移出

2.6.2 优先权

优先权调度：发送最高优先权的分组

多类，不同类别有不同的优先权

类别依赖于标记或者其他头部字段

先传优先权高的分组，相同优先权分组中按照FIFO传输

2.6.3 Round Robin scheduling

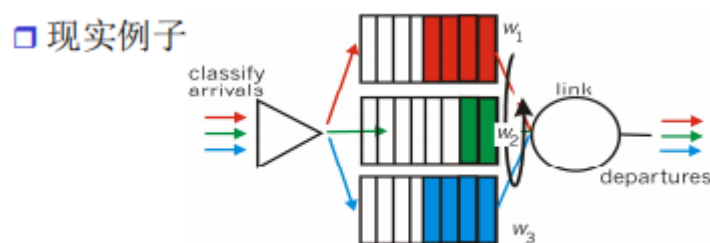
多类

循环扫描不同类型的队列，发送完一类的一个分组，再发送下一个类的一个分组，循环所有类

2.6.4 Weighted Fair Queuing

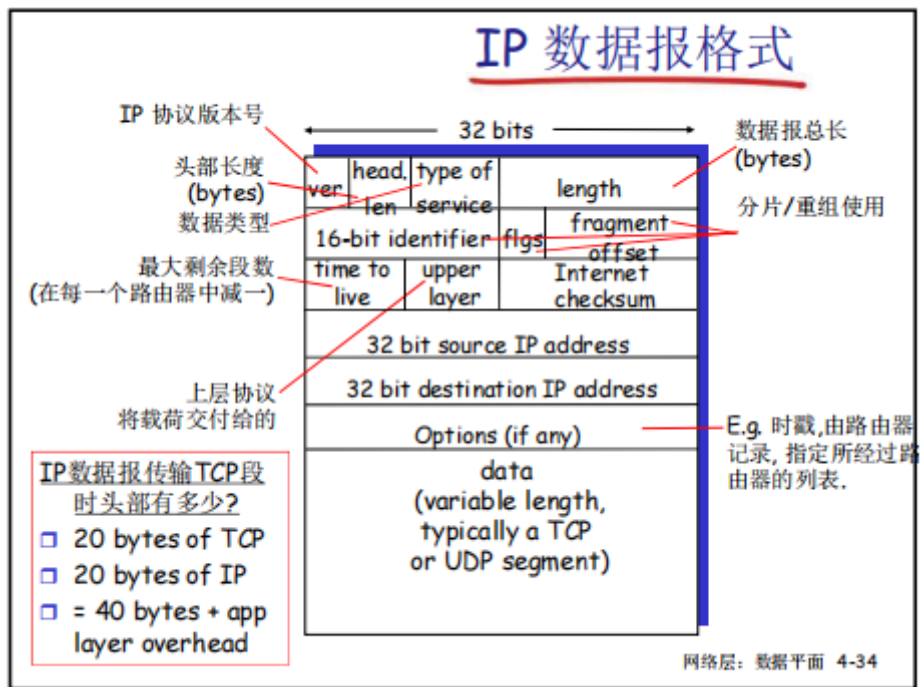
一般化的Round Robin

在一段时间内，每个队列得到的服务时间是： $W_i / \sum(W_i) * t$ ，和权重成正比



3.IP

3.1 IPv4数据报格式



3.2 IP分片和重组

大的IP数据报在网络上被分割成若干片：这些片具有相同的ID，不同的 offset，如果不是最后一个片，flag=1，否则 flag=0。

分组的重新组合只在最终的目标主机中进行。

IP头部的信息用于标识，排序相关分片。

3.3 IP编址

- IP地址：32位标识，对主机或者路由器的接口编址
- 接口：主机/路由器和物理链路的连接处
 - 路由器 and 主机可能有多个接口，而IP地址和每一个接口相关联
- 一个IP地址和一个接口相关联

3.4 子网

IP地址=子网部分（高位）+主机部分（低位）

一个子网内的节点的IP地址的高位是相同的，这些节点构成的网络的一部分叫做子网。

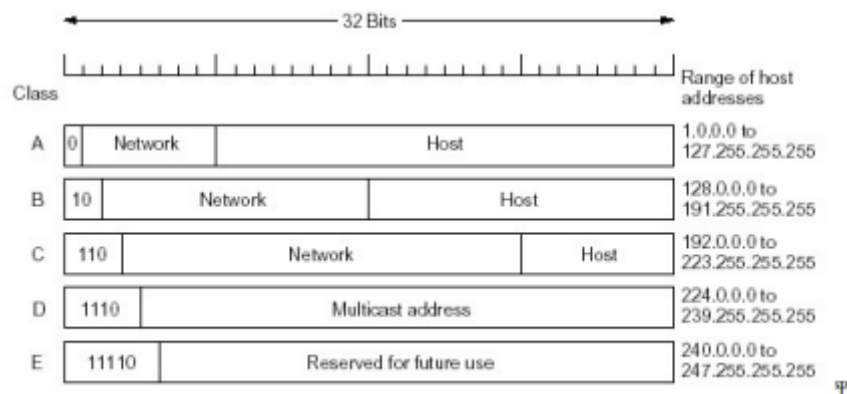
子网类的主机可以在物理上相互直接到达。

每一个网络孤岛都可以称为一个 subnet。

3.5 IP地址分类

3.5.1 有类网络分类

- Class A: 126 networks , 16 million hosts
- Class B: 16382 networks , 64 K hosts
- Class C: 2 million networks , 254 host
- Class D: multicast
- Class E: reserved for future



3.5.2 特殊的IP地址

- 子网部分：全为0——本网络
- 主机部分：全为0——本主机
- 主机部分：全为1——广播地址，这个网路的所有主机

3.5.3 内网（专用）IP地址

不会被用作公用地址来分配，只在局部网络中有意义，区分不同的设备。路由器不对目标地址是专用地址的分组进行转发。

专用地址范围：

Class A 10.0.0.0—10.255.255.255 MASK 255.0.0.0

Class B 172.16.0.0—172.31.255.255 MASK 255.255.0.0

Class C 192.168.0.0—192.168.255.255 MASK 255.255.255.0

3.5.4 CIDR（无类域间路由）

子网部分可以在任意的位置，变长。

子网掩码 subnet mask：32位，1表示子网部分，0表示主机部分。

表示方法：

- 32位01位串
- /#：#表示前为1的位数，即子网部分

3.6 IP地址的获取

3.6.1 主机获取IP地址的方法

- 系统管理员将地址配置在一个文件中
- DHCP: Dynamic Host Configuration Protocol 从服务器中动态获得一个IP地址。 plug and play

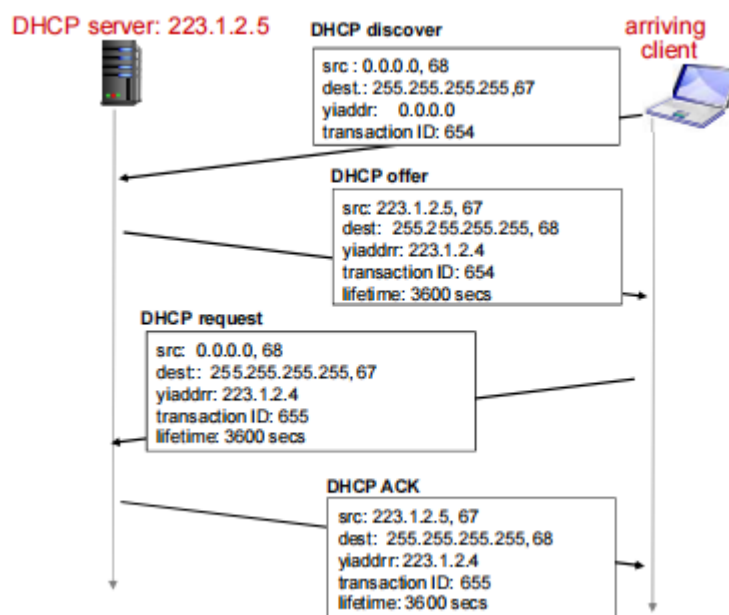
3.6.2 DHCP

主机在加入网络时，动态地从服务器中获得IP地址：

- 可以更新对主机在用IP地址的租期
- 重新启动时，允许重新使用以前用过的IP地址
- 支持移动用户加入网络（短期在网）

DHCP工作概况：

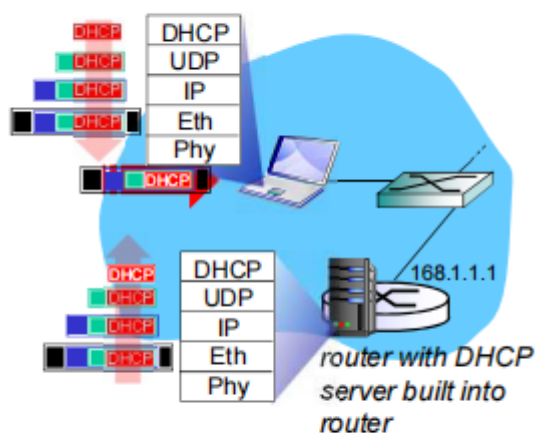
- 主机广播“DHCP discover” 报文[可选]
- DHCP 服务器用 “DHCP offer”提供报文响应[可选]
- 主机请求IP地址：发送 “DHCP request” 报文
- DHCP服务器发送地址：“DHCP ack” 报文



DHCP返回：

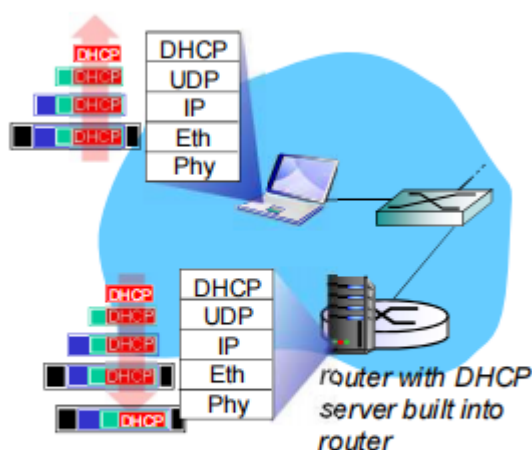
- IP地址
- 第一跳路由器的IP地址（默认网关）
- DNS服务器的域名和IP地址
- 子网掩码（指示地址部分的网络号和主机号）

DHCP: 实例



- ❖ 联网笔记本需要获取自己的IP地址，第一跳路由器地址和DNS服务器：采用DHCP协议
- ❖ DHCP 请求被封装在UDP段中，封装在IP数据报中，封装在以太网的帧中
- ❖ 以太网帧在局域网范围内广播 (dest: FFFFFFFF)，被运行DHCP服务的路由器收到
- ❖ 以太网帧解封装成IP，IP解封装成UDP，解封装成DHCP

DHCP: 实例



- ❑ DHCP服务器生成DHCP ACK，包含客户端的IP地址，第一跳路由器的IP地址和DNS域名服务器的IP地址
- ❖ DHCP服务器封装的报文所在的帧转发到客户端，在客户端解封装成DHCP报文
- ❖ 客户端知道它自己的IP地址，DNS服务器的名字和IP地址，第一跳路由器的IP地址

3.6.3 获取网络中子网的方法

从ISP获得地址块中分配一个小地址块

层次编址：路由聚集 route aggregation (详见ref子网和网关)

3.6.4 ISP获取地址的方法

ICANN: Internet Corporation for Assigned Names and Numbers

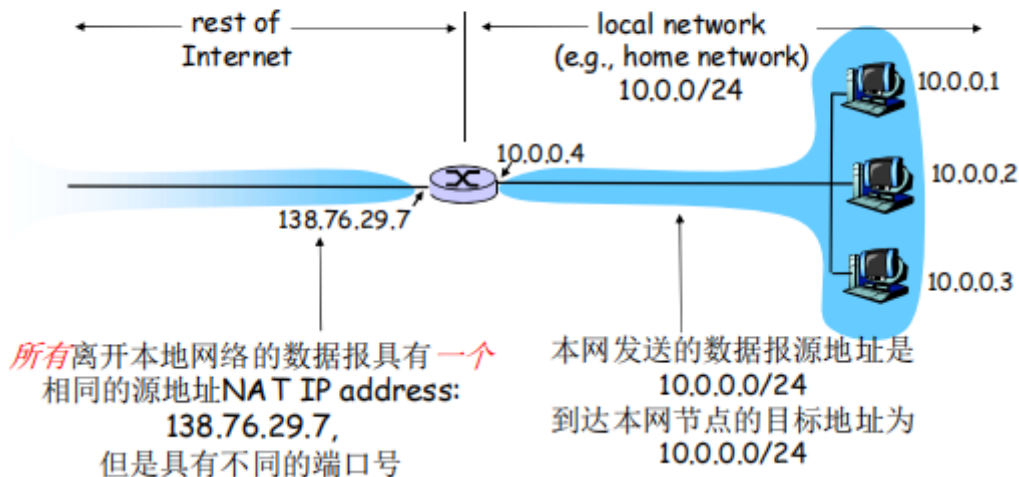
- 分配地址
- 管理DNS
- 分配域名，解决冲突

3.7 NAT

本地网络只有一个有效IP地址：

- 不需要从ISP分配地址，用一个IP地址表示所有的设备
- 可以在局域网改变设备的地址情况下而无须通知外界
- 可以改变ISP而不需要改变内部的设备地址
- 局域网内部的设备没有明确的地址——对外不可见：安全

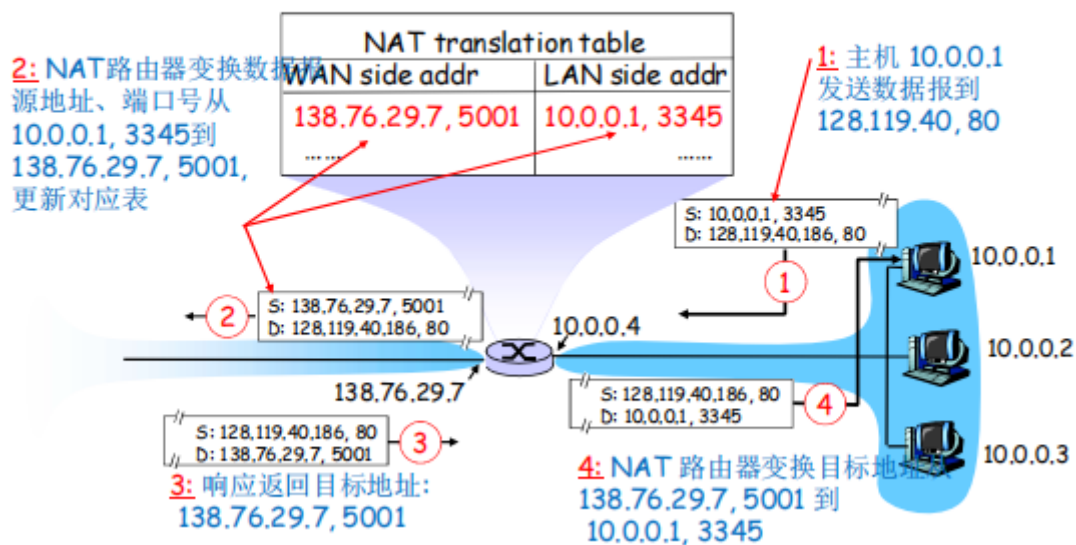
NAT: Network Address Translation



NAT的实现：

- 外出数据包：将源地址和端口号替换成NAT IP地址和新的端口号，目标IP和端口不变
- 记住每一个转换替换对：在NAT转换表中
- 进入数据包：替换目标IP地址和端口号，使用NAT表中的映射

NAT: Network Address Translation



NAT存在的问题：

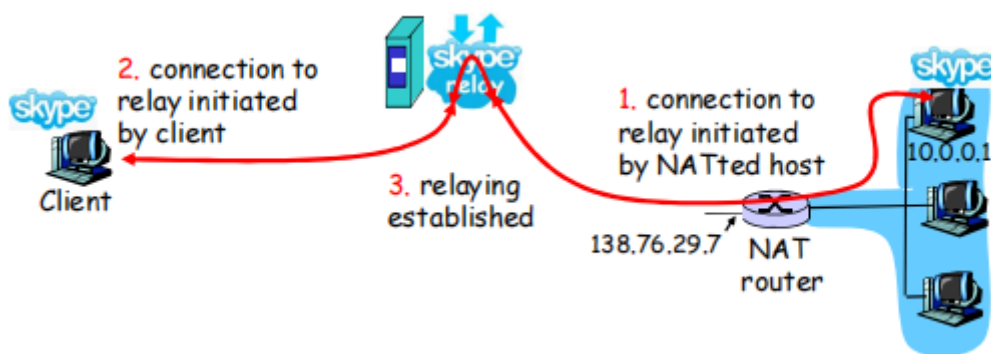
- 路由器只应该对第三层做信息处理，而这里对端口号（4层）作了处理

- 违反了端到端原则：复杂性放到网络边缘；因为使用NAT会导致网络核心中的复杂性
- 地址不时地变化：P2P需要考虑
- 外网的机器无法主动连接到内网的机器上
- 地址短缺问题可以被IPv6解决
- NAT穿越：如果客户端需要连接在NAT后面的服务器，如何操作

NAT穿越问题：客户端需要连接内网的一台主机，但是整个网络只有一个外部可见地址。

方案：

1. 静态配置NAT：转发进来的对服务器特定端口连接请求
2. UPnP 协议、IGD 协议：允许主机可以获得网络的公共IP地址，列举存在的端口映射，增/删端口映射
3. 中继：NAT后面的服务器建立和中继的连接，外部的客户端连接到中继，中继建立起桥梁

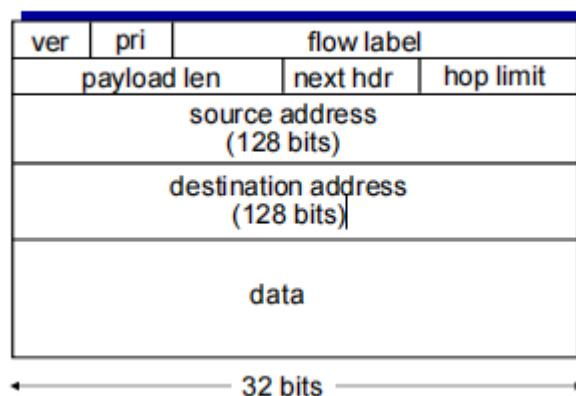


3.8 IPv6

启用IPv6的原因：32位地址空间很快被用完，并且原先的头部有欠缺

IPv6数据报格式：

- 固定的40字节头部
- 数据报传输过程中，不允许分片



网络层：数据平面

Priority：标示流中数据报的优先级

Next header：标示上层协议

Checksum：被移除掉，降低在每一段中的处理速度

Options：允许，但是在头部之外，被 Next Header 字段标示

- 附加了报文类型, 如 Packet Too Big
- 多播组管理功能

从IPv4到IPv6的过渡 (详见ref: IPv4—IPv6的过渡技术)

4.通用转发和SDN

网络层功能: 转发、路由 (控制平面数据平面相互配合) ——传统方法: 垂直集成、分布式实现、每个路由器都包含数据平面和控制平面。

4.1 传统方法

特点: 种类繁多, 网络设备更改困难, 控制平面分布式实现, 行为僵化、升级管理很难

- 垂直集成: 硬件、OS、Protocol都私有
- 每台设备包含所有层面的功能: 控制功能分布式实现
- 控制逻辑固化: 按照固定方式工作

问题:

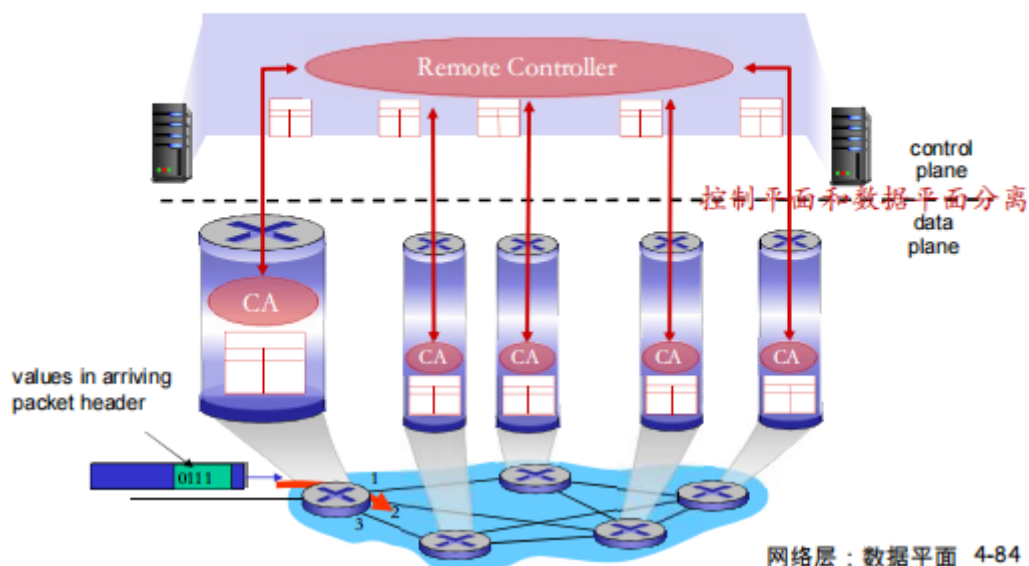
- 昂贵
- 无法改变工作逻辑
- 管理困难
- 更新困难

——2005年开始考虑重新处理控制平面的实现方式:

- 集中: 远程的控制器集中实现控制逻辑
- 远程: 数据平面和控制平面的分离

4.2 SDN: 逻辑上集中的控制平面

一个不同的 (通常是远程) 控制器和CA交互, 控制器决定分组转发的逻辑 (可编程), CA所在设备执行逻辑。



远程计算流表, 通过南向接口发送到每台分组交换设备, 增加匹配规则和对应的操作。

优点：

- 可以**随时更改升级**
- 可以让多个分组交换机厂商参与竞争——**更好的生态**
- 可以在网络OS上运行**网络应用**

主要思路：

- 网络设备数据平面和控制平面**分离**
- 控制平面功能在**数据交换设备之外实现**
- 数据平面——分组交换机：可以**转发、丢弃、拷贝、泛洪、阻塞**
- 分组交换机**通过流表**进行动作（OpenFlow）
- 控制平面——**控制器+网络应用**

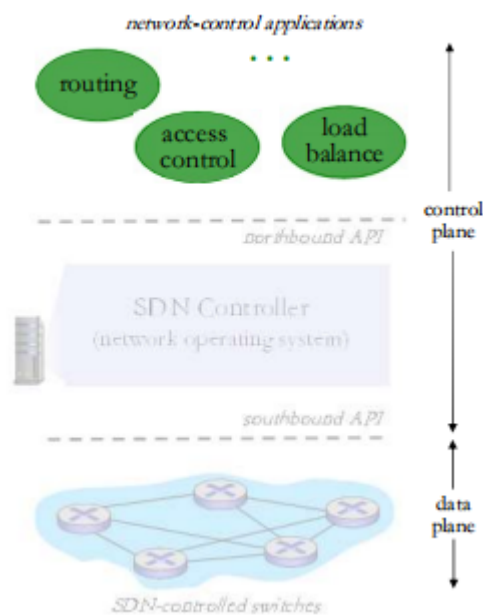
分离的优势：

- 水平集成控制平面的开放实现创造出好的产业生态
- 集中式实现控制逻辑，网络管理容易
- 基于流表的匹配+行动的工作方式允许“**可编程的**”分组交换机

传统方法流量工程的问题：

- 自主选择路径，只能通过改变链路代价来实现，非常困难且无法按自己的需求实现
- 两路分拆负载均衡，无法完成，只能完全更换新的路由算法
- 不同流量走不同的路由，无法完成，只能基于目标进行转发

4.3 SDN架构



4.3.1 数据平面交换机

- 基于南向API，SDN控制器访问基于流的交换机
- 使用流表进行匹配
- 采用硬件实现通用转发
- 定义了控制器的协议

4.3.2 SDN控制器（网络OS）

- 维护网络状态信息
- 通过上面的北向API和网络控制应用交互
- 通过下面的南向API和网络交换机交互
- 逻辑上集中，物理上可以分布式实现

4.3.3 控制应用

- 控制的大脑：采用下层提供的服务实现网络功能：路由、接入控制、防火墙、负载均衡
- 非绑定：可以适配不同公司的设备，生态健康

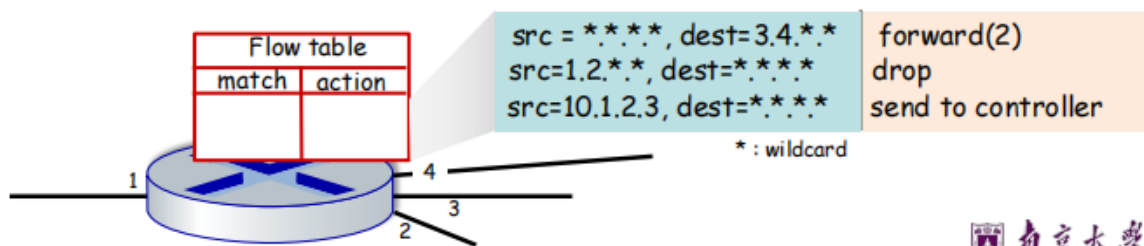
通用转发和SDN过程：查看分组条目属性、匹配流表表项（头部、计数、操作等等）、执行对应操作。

4.4 OpenFlow数据平面抽象

流：由分组（帧）头部字段所定义

通用转发：简单的分组处理规则

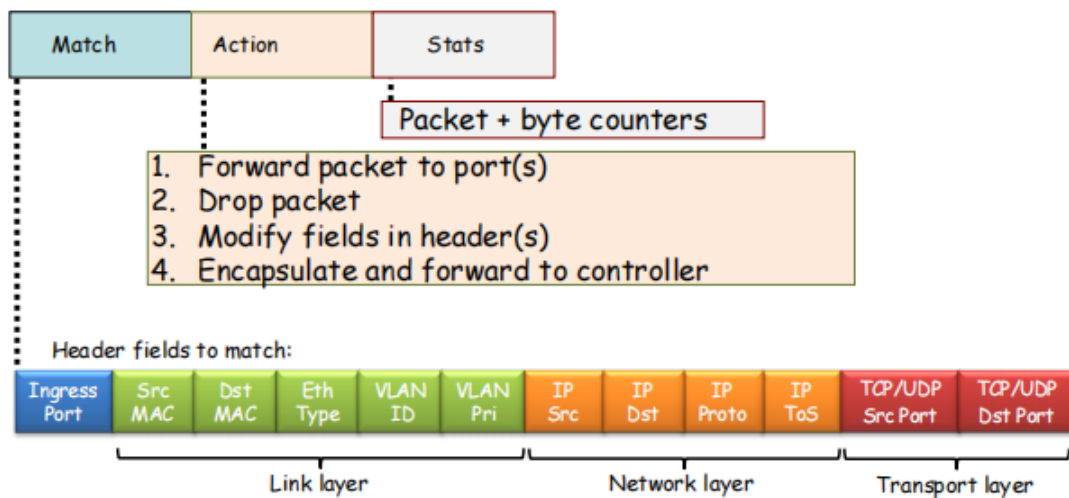
- 模式：将分组头部字段和流表进行匹配
- 行动：对于匹配上的分组，可以是丢弃、转发、修改、将匹配的分组发送给控制器
- 优先权Priority：几个模式匹配了，优先采用哪个，消除歧义
- 计数器Counters: #bytes 以及 #packets



路由器中的流表定义了路由器的匹配+行动规则

流表的表项结构：

1. Rule（规则、模式）：十几个字段
2. Action（行动）：各种操作
3. Stats（状态、计数）：基于包、基于字节



OpenFlow抽象:

match+action (匹配+行动模式) : 统一化各种网络设备提供的功能

Router

- **match:** longest destination IP prefix
- **action:** forward out a link

Firewall

- **match:** IP addresses and TCP/UDP port numbers
- **action:** permit or deny

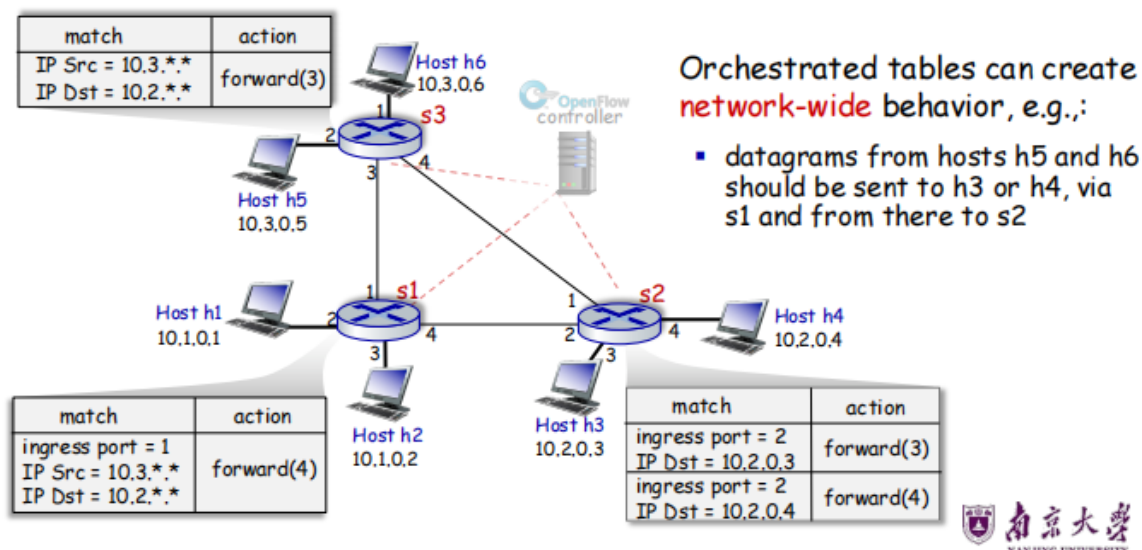
Switch

- **match:** destination MAC address
- **action:** forward or flood

NAT

- **match:** IP address and port
- **action:** rewrite address and port

示例:



4.5 中间盒